PRACTICAL 4 (DAA)

Factorial using Iterative and Recursive Methods

```
#include <iostream>
#include <chrono>
using namespace std;

Iterative method
unsigned long long factorialIterative(int n)
{
    unsigned long long fact = 1;

    for (int i = 1; i <= n; i++)
    {
        fact = fact * i;
    }

    return fact;
}

Recursive method
unsigned long long factorialRecursive(int n)
{
    Base condition
    if (n == 0 || n == 1)
        return 1;

    Recursive call
    return n * factorialRecursive(n - 1);
}

int main()
{
    int n;

    cout << "Enter a non-negative integer: ";
    cin >> n;

    if (n < 0)
    {
        cout << "Invalid input!";
        return 0;
    }

    Iterative factorial
    auto start1 = chrono::high_resolution_clock::now();

    unsigned long long result1 = factorialIterative(n);

    auto end1 = chrono::high_resolution_clock::now();

    Recursive factorial
    auto start2 = chrono::high_resolution_clock::now();

    unsigned long long result2 = factorialRecursive(n);

    auto end2 = chrono::high_resolution_clock::now();

    Calculate execution time
    auto time1 = chrono::duration_cast<chrono::nanoseconds>
                (end1 - start1);

    auto time2 = chrono::duration_cast<chrono::nanoseconds>
                (end2 - start2);

    Display results
```

```
    cout << "\nFactorial of " << n << ":\n";

    cout << "Iterative Result : " << result1 << endl;
    cout << "Iterative Time    : " << time1.count() << " ns" << endl;

    cout << "\nRecursive Result : " << result2 << endl;
    cout << "Recursive Time    : " << time2.count() << " ns" << endl;

    return 0;
}
```